Composing cellular automata

Simon Frost

Table of contents

1 Overview 1

2 The composite of two automata 2

3 The category laws 2

4 Memory sets can shrink 3

5 Composing across alphabets 3

6 Order matters on a non-abelian group 4

7 A composite on CellularAutomataCore 4

1 Overview

Applying one cellular automaton and then another is again a cellular automaton, with a larger memory set and a local rule built from the two local rules (Castillo-Ramirez et al. 2026). Composition is associative and has identities, so cellular automata over a group form a category. This vignette composes automata over rings, tori and a non-abelian group, between different alphabets, and runs a composite on CellularAutomataCore.

```
ENV["GKSwstype"] = "100"
using CategoricalCellularAutomata
using CellularAutomataCore: CellularAutomataCore, Life, Torus, CAProblem, solve
using Random
```

2 The composite of two automata

If τ has memory set `S` and local rule μ, and σ has memory set `T` and local rule ν, then σ(τ(x)) at a cell `g` reads τ(x) at the cells `t g`, each of which reads `x` at the cells `s t g`. The composite is the cellular automaton with memory set `{s t : s ∈ S, t ∈ T}` whose local rule applies μ once for each `t` and then ν. `compose(τ, σ)` builds it, listing the automata in the order they are applied (as in a pipeline), and `σ ∘ τ` is the same in the order of function composition.

```
Z = CyclicGroup(16)
B = FinAlphabet(0:1)
rule90 = CellularAutomaton(Z, [15, 1], p -> xor(p...); dom=B)
rule150 = CellularAutomaton(Z, [15, 0, 1], p -> xor(p...); dom=B)
both = compose(rule90, rule150)
memory_set(both)
```

```
5-element Vector{Int64}:
 14
  0
 15
  1
  2
```

The memory set is `{-2, …, 2}`, written modulo 16. The composite is one automaton, and it agrees with applying the two in turn:

```
x = Configuration(Z, rand(Xoshiro(1), 0:1, 16))
(both(x) == rule150(rule90(x)), both == rule150 ∘ rule90)
```

```
(true, true)
```

3 The category laws

Composition is associative, and the identity automaton, whose memory set is the identity element alone, is a unit on either side. The equalities are checked with `==`, which compares local rules on every pattern over the memory sets, so they hold for every configuration of every size of ring:

```
X = dom(rule90)
rule30 = CellularAutomaton(Z, [15, 0, 1], p -> p[1] ⊻ (p[2] | p[3]); dom=B)
(compose(compose(rule90, rule150), rule30) == compose(rule90, compose(rule150,
rule30)),
 compose(id(X), rule30) == rule30 == compose(rule30, id(X)))
```

```
(true, true)
```

4 Memory sets can shrink

The memory set of a composite is a bound, not always the set of cells the composite depends on. Rule 90 applied twice reads `x(g-2)`, `x(g)` twice and `x(g+2)`, and the two copies of `x(g)` cancel in the exclusive or. `minimal_neighbourhood` keeps only the cells the rule depends on:

```
twice = rule90^2
(memory_set(twice), memory_set(minimal_neighbourhood(twice)))
```

```
([14, 0, 2], [14, 2])
```

So rule 90 applied twice is rule 90 with its neighbours twice as far away, and `==` confirms it:

```
twice == CellularAutomaton(Z, [14, 2], p -> xor(p...); dom=B)
```

```
true
```

By the same argument, rule 90 applied `2^k` times is rule 90 at distance `2^k`. This is why it draws Pascal's triangle modulo 2.

5 Composing across alphabets

The codomain of the first automaton must be the domain of the second, but the alphabets can differ. Here an epidemic on a ring, with susceptible, infected and removed plants, is followed by a survey that records only whether each plant shows symptoms:

```
SIR = FinAlphabet([:S, :I, :R])
epidemic = CellularAutomaton(Z, [15, 0, 1],
                             p -> p[2] == :S && :I in (p[1], p[3]) ? :I :
                                  p[2] == :I ? :R : p[2]; dom=SIR)
survey = CellularAutomaton(Z, [0], p -> Int(only(p) == :I); dom=SIR, codom=B)
observed = compose(epidemic, survey)
```

```
CellularAutomaton over CyclicGroup(16) with memory set [15, 0, 1]: [:S, :I, :R] →
[0, 1]
```

The composite goes from `{S, I, R}` to `{0, 1}`. The other order is refused, because the survey's codomain is not the epidemic's domain:

```
try
    compose(survey, epidemic)
catch err
    sprint(showerror, err)[1:60] * "..."
end
```

```
"DomainMismatchError: composing cellular automata expected Fu..."
```

6 Order matters on a non-abelian group

On the symmetric group S3 the memory set of a composite is `{s t : s ∈ S, t ∈ T}`, with `s` on the left, and `s t` is not `t s`:

```
perms = [[1, 2, 3], [2, 1, 3], [1, 3, 2], [3, 2, 1], [2, 3, 1], [3, 1, 2]]
S3 = FiniteGroup(perms, (a, b) -> a[b])
τ = CellularAutomaton(S3, [2], only; dom=B)        # x ↦ x(2 g)
σ = CellularAutomaton(S3, [3], only; dom=B)        # x ↦ x(3 g)
(memory_set(compose(τ, σ)), mul(S3, 2, 3), mul(S3, 3, 2))
```

```
([5], 5, 6)
```

The composite reads `x(2 3 g)`: `σ` reads `τ(x)` at `3 g`, and `τ` reads `x` at `2 (3 g)`. Applying the two in turn agrees:

```
z = Configuration(S3, [1, 0, 0, 1, 1, 0])
compose(τ, σ)(z) == σ(τ(z))
```

```
true
```

7 A composite on CellularAutomataCore

Life is a cellular automaton over the torus group. Composed with itself it is a single automaton whose memory set is the 5×5 square, which runs on CellularAutomataCore like any other rule:

```
B8 = FinAlphabet(Int8[0, 1])
life = CellularAutomaton(Life(), Torus(32, 32); alphabet=B8)
life2 = life^2
length(memory_set(life2))
```

```
25
```

One step of the composite is two steps of Life:

```
u0 = Int8.(rand(Xoshiro(2), 0:1, 32, 32))
solve(CAProblem(life2, u0, 10)).u[end] == solve(CAProblem(Life(), u0, 20)).u[end]
```

```
true
```

Its local rule evaluates Life's local rule nine times per cell, so it is not a faster way to run Life. Its use is that the composite is an object of its own, which can be compared, minimised, paired with other automata and passed on. The next vignette pairs automata, which is how multi-layer models are built.

Castillo-Ramirez, Alonso, Alejandro Vazquez-Aceves, and Angel Zaldivar-Corichi. 2026. *A Categorical Framework for Cellular Automata*. https://arxiv.org/abs/2602.04049.